

Cooperative Robotic Transport using Reinforcement Learning: A Comparative Planning Analysis

1st Aditya Uday Abhang

Autonomous and Connected Systems (ACS)

Purdue University

West Lafayette, IN

aabhang@purdue.edu

2nd Jaemin Cho

Autonomous and Connected Systems (ACS)

Purdue University

West Lafayette, IN

cho696@purdue.edu

Abstract—This paper introduces a novel formulation for cooperative robotic transport. We model two agents carrying a rigid object, represented as a line segment, through a constrained indoor environment. This task is framed as a Markov Decision Process to manage the coupled state-space and complex collision constraints. We then implement and systematically compare classical model-based planning algorithms, specifically Value Iteration and Policy Iteration, against a model-free reinforcement learning approach, Q-Learning. The analysis focuses on the trade-offs between computational cost, optimality of the resulting path, and the learning convergence speed for this coupled-agent navigation problem.

Index Terms—cooperative robotics, multi-agent systems, path planning, Markov Decision Process, reinforcement learning

I. PROBLEM STATEMENT

While standard robot navigation research often focuses on single-agent pathfinding or multi-agent collision avoidance, cooperative manipulation tasks involving physically coupled agents remain a significant challenge. This project addresses the problem of cooperative transport, specifically modeling two robots carrying a rigid ‘mattress’ (represented as a line segment) through a constrained, 2D indoor environment. The core challenge stems from the coupled state-space and complex action constraints this task creates, rendering standard single-agent algorithms ineffective.

The objective is to formulate this cooperative transport task as a Markov Decision Process (MDP) and determine an optimal navigation policy. This work will systematically compare the performance, optimality, and computational trade-offs of classical model-based planners (Value Iteration, Policy Iteration) against a model-free, learning-based approach (Q-Learning) for this novel, coupled-state navigation problem.

II. LITERATURE REVIEW

The problem of navigating a rigid object through a cluttered environment using multiple agents—classically known as the “Piano Movers’ Problem”—bridges the domains of geometric path planning, multi-agent coordination, and reinforcement learning (RL). The literature illustrates a progression from exact geometric formulations in configuration space to approximate, learning-based solutions capable of handling complex dynamic constraints.

A. Foundational Geometric Models: The Piano Movers’ Problem

The theoretical foundation of this project rests on the generalized mover’s problem. Schwartz and Sharir [1] formally defined the Piano Movers’ problem, presenting an algorithm to determine the existence of a continuous collision-free path for a rigid polygonal body amidst static polygonal barriers. Their work established the rigorous geometric constraints required to navigate tight spaces, decomposing the free space into connected cells to construct a connectivity graph [1].

Central to solving such problems is the concept of Configuration Space (C-Space), which transforms the problem of moving a dimensioned object into moving a point within a higher-dimensional manifold. Red and Truong-Cao [2] demonstrated the practical application of this approach by mapping Cartesian workspace obstacles into joint space, allowing for the identification of collision-free regions for manipulator planning. This transformation is critical for the current project, as it justifies modeling the multi-agent mattress system as a single rigid body with a specific footprint in the grid map, effectively reducing the coordination problem to planning in a constrained $SE(2)$ state space.

B. Algorithmic Frameworks: Planning and Reinforcement Learning

While geometric methods provide exact solutions, they often suffer from computational intractability in high dimensions. To address this, probabilistic and learning-based methods have been developed. Bretl et al. [3] explored multi-step motion planning in highly constrained environments (e.g., free-climbing robots), utilizing Probabilistic Roadmaps (PRM) to capture connectivity in high-dimensional configuration spaces. Their work highlights the necessity of sample-based approaches when analytical solutions become infeasible due to complex kinematic constraints [3].

For dynamic decision-making, the problem is often formulated as a Markov Decision Process (MDP). Sutton and Barto [4] provide the definitive framework for this approach, detailing dynamic programming methods such as Value Iteration and Policy Iteration. These algorithms offer guarantees of optimality given a known model of the environment but

are computationally expensive for large state spaces [4]. Conversely, Q-Learning allows for model-free control, enabling agents to learn optimal policies through interaction with the environment [4]. This distinction forms the basis of the comparative analysis in this project, specifically regarding the trade-off between the sample efficiency of model-based planning and the flexibility of model-free RL.

C. Multi-Robot Cooperative Transport

The extension of these concepts to multi-agent systems introduces additional complexity regarding coordination and constraints. Zhang et al. [5] addressed the specific challenge of multi-robot cooperative object transportation using Decentralized Deep Reinforcement Learning (DDRL). They proposed a scheme where individual robots, equipped with Deep Q-Network (DQN) controllers, learn to transport oversized objects without explicit communication [5]. Their research underscores the difficulty of maintaining formation and the physical constraints of the transported object, effectively managing the system's dynamics through learned policies rather than rigid geometric planning [5].

D. Synthesis

The literature reveals a dichotomy between the exact, geometrically rigorous methods of Schwartz and Sharir [1] and the approximate, scalable approaches of modern RL [4, 5]. While C-Space approaches [2] provide the necessary theoretical constraints for rigid body movement, they struggle with the "Curse of Dimensionality" inherent in multi-agent systems. Conversely, RL methods like Q-Learning [4] and decentralized Deep RL [5] offer robust mechanisms for navigating these high-dimensional spaces but require extensive training and careful reward shaping. This project synthesizes these views by applying foundational algorithms (VI, PI) to a scaled grid-based model of the Piano Movers' problem, demonstrating how rigid body constraints can be effectively managed in a discrete state space while highlighting the computational limits that necessitate heuristic or learning-based extensions [4, 5].

III. PROBLEM FORMULATION

To compare these algorithms, we first formally define the problem as a Markov Decision Process (MDP), which is characterized by a tuple (S, A, R, P, γ) , where P is the transition model and γ is a discount factor.

A. Models

- **Markov Decision Process (MDP):** The foundational framework we use to model the decision-making problem. We assume the environment is fully observable (the agent knows its state s) and the transitions are deterministic (Action 'A' in state 'S' always leads to state 'S').
- **Value Iteration (VI) Policy Iteration (PI):** These are model-based, dynamic programming algorithms. They require complete knowledge of the entire MDP (all states S , actions A , rewards R , and transitions P) in advance to find the provably optimal policy.

- **Q-Learning:** This is a model-free, temporal-difference reinforcement learning algorithm. It does not require prior knowledge of the rewards or transitions. It learns the optimal policy by exploring the environment through trial-and-error, updating its estimate of an action's value (Q -value) based on the rewards it receives.

B. Environment

- **Space:** A 2D grid of size $N \times M$ cells, representing a small map of a house.
- **Cell Types:** Each cell in the grid is one of the following:
 - **Traversable:** An empty space where the robots (and mattress) can move.
 - **Wall/Obstacle:** An impassable barrier.
 - **Start State (s_{start}):** The initial configuration (position and orientation) of the mattress.
 - **Goal State (s_{goal}):** The target configuration.

The environment is constructed from the provided floor plan image. The image is resized to 100×100 using OpenCV, converted to grayscale, and binarized using Otsu thresholding to obtain a grid map consisting of free cells (value 0) and obstacle cells (value 1).

The start and goal positions are marked with different colors in the original image. The function `get_color_centroid` creates a mask for each color, finds the largest contour, and computes its centroid, which is converted into grid coordinates. If a marker is not found, the default values (70, 20) for the start position and (45, 80) for the goal position are used.

C. States (S)

A state s must capture the entire configuration of the coupled robot-mattress system. A simple (x, y) coordinate is insufficient.

The agent is modeled as a one-dimensional rigid body (the mattress) of length $L = 5$ cells. A state is defined as

$$s = (x, y, \theta),$$

where (x, y) is the center cell on the grid, and $\theta \in \{0, \dots, 7\}$ is a discrete orientation in 45° increments (eight possible directions). For each state, the front end point (x_f, y_f) and the back end point (x_b, y_b) of the mattress are computed as

$$\theta_{rad} = \theta \times 45^\circ, \quad (1)$$

$$\Delta x = \frac{L}{2} \cos \theta_{rad}, \quad \Delta y = \frac{L}{2} \sin \theta_{rad}, \quad (2)$$

$$(x_f, y_f) = (x + \Delta x, y + \Delta y), \quad (3)$$

$$(x_b, y_b) = (x - \Delta x, y - \Delta y). \quad (4)$$

A state is considered *valid* if the center and both end-points lie inside the grid and all of them are free cells (non-obstacles). The method constructs the state set S by enumerating all states that satisfy this condition.

D. Actions (A)

Actions are defined for the entire system, not for the individual robots. An action a is a command to move the mattress from its current state s to a new state s' .

- Actions are defined for the entire system, not for the individual robots. An action a is a command to move the mattress from its current state s to a new state s' .
 - Translate North: $(x, y, \theta) \rightarrow (x, y - 1, \theta)$
 - Translate South: $(x, y, \theta) \rightarrow (x, y + 1, \theta)$
 - Translate East: $(x, y, \theta) \rightarrow (x + 1, y, \theta)$
 - Translate West: $(x, y, \theta) \rightarrow (x - 1, y, \theta)$
 - Rotate: $(x, y, \theta) \rightarrow (x, y, 1 - \theta)$ (i.e., flip from horizontal to vertical or vice-versa, rotating around the center point (x, y)).
- Action Validity: An action is invalid and cannot be taken if the resulting state s' would place any part of the line segment (mattress) onto a Wall/Obstacle cell. This collision checking is the central constraint of the problem.

E. Rewards (R) / Cost

The reward function $R(s, a, s')$ guides the learning process. We design it to achieve the goal safely and efficiently, consistent with reward-shaping techniques seen in the literature [4, 5].

The goal region is defined as the set of states whose center is within a radius of four cells from the goal position (x_g, y_g) . The reward function is

$$R(s) = \begin{cases} +100, & \text{if } \sqrt{(x - x_g)^2 + (y - y_g)^2} < 4, \\ -1, & \text{otherwise.} \end{cases} \quad (5)$$

Thus, the agent receives a reward of -1 at every step until it enters the goal region, which encourages policies that minimize path length. The maximum episode length is limited to 400 steps to prevent infinite loops.

IV. SOLUTION METHODS

A. Value Iteration

Value iteration is a dynamic programming method that approximates the optimal value function $V^*(s)$ by repeatedly applying the Bellman optimality equation when both the transition model and the reward function are known. In this project, we use discount factor $\gamma = 0.95$.

The update rule is

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} [R(s') + \gamma V_k(s')], \quad s' = \text{step}(s, a), \quad (6)$$

and it is applied to all states. We stop when the maximum change $\Delta = \max_s |V_{k+1}(s) - V_k(s)|$ is smaller than a threshold $\theta = 0.1$.

After convergence, the policy is extracted via one-step look-ahead:

$$\pi(s) = \arg \max_a [R(s') + \gamma V(s')]. \quad (7)$$

B. Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement. Starting from an initial policy π , we repeat:

- 1) **Policy Evaluation:** For a fixed policy π ,

$$V(s) \leftarrow R(s') + \gamma V(s'), \quad s' = \text{step}(s, \pi(s)), \quad (8)$$

is iterated multiple times to approximate the value function under policy π . In our implementation, we perform 10 full sweeps per evaluation phase.

- 2) **Policy Improvement:** For each state,

$$\pi_{\text{new}}(s) = \arg \max_a [R(s') + \gamma V(s')], \quad (9)$$

and the policy is updated to π_{new} . If the policy no longer changes, the algorithm terminates.

In theory, policy iteration converges in fewer iterations than value iteration, but each iteration is more expensive due to policy evaluation, so the total computation can be higher.

C. Q-Learning and Reward Shaping

Q-learning is a model-free reinforcement learning algorithm that does not require transition probabilities. The state-action value function $Q(s, a)$ is stored in a table, and we use the following off-policy TD update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (10)$$

1) *Distance-Based Reward Shaping:* With a purely sparse reward structure, the agent rarely finds the goal and learning becomes extremely slow. To improve this, we add a reward shaping term based on the change in distance to the goal.

Let d_{old} and d_{new} be the Euclidean distances from the current and next state to the goal, respectively. We define

$$r_{\text{shape}} = \lambda(d_{\text{old}} - d_{\text{new}}), \quad \lambda = 0.5, \quad (11)$$

and use

$$r = R(s') + r_{\text{shape}} \quad (12)$$

in the TD update. Moving closer to the goal yields a small bonus, while moving farther away yields a penalty, which creates a smoother gradient in the value function.

For fair comparison with VI and PI, the episode return logged for plots uses only the original reward $R(s')$. For terminal goal states, we remove the bootstrapping term and use r itself as the TD target.

2) *Exploration and Learning Schedule:* All episodes start from the same initial state. The start state is chosen as the valid state closest to the desired start position, with orientation $\theta = 0$.

We use an ε -greedy exploration strategy with the following parameters:

- Initial $\varepsilon = 1.0$, minimum $\varepsilon_{\text{min}} = 0.05$.
- Initial learning rate $\alpha = 0.5$, minimum $\alpha_{\text{min}} = 0.1$.
- Both ε and α are multiplied by 0.9998 after each episode.

Each episode is limited to 400 steps, and we count how many episodes reach the goal to analyze the success rate. Learning is performed for a total of 25,000 episodes.

TABLE I
QUANTITATIVE COMPARISON OF THE THREE ALGORITHMS

Algorithm	Time (s)	Iter/Episodes	Total Reward	Steps
Value Iteration	47.91	72	19	82
Policy Iteration	202.32	67	19	82
Q-Learning	53.21	25000	19	82

V. EXPERIMENTAL SETUP

All algorithms are run on the same 100×100 grid map, with the same start/goal state, reward function, and discount factor. After creating the environment, we compute the full set of valid states using `get_valid_states`, and VI/PI perform full sweeps over this set. Q-learning uses the same state set as indices of the Q-table, but updates occur only for states actually visited during episodes.

The comparison metrics are:

- Computation time (seconds).
- Number of iterations: VI/PI iterations or Q-learning episodes.
- Total reward: cumulative reward obtained when following the greedy policy from the start state to the goal.
- Number of steps: length of the resulting path.
- For Q-learning, success rate and 500-episode rolling average return.

The paths generated by the final policy (or learned Q-table) are visualized on the map using the trajectories of the front and back endpoints to check for collisions and the naturalness of the motion.

VI. EXPERIMENTAL RESULTS

A. Quantitative Comparison

The main results of the three algorithms are summarized in Table I.

All three algorithms achieve a path of length 82 and a total reward of 19, indicating convergence to the same (or very similar) optimal policy. Although policy iteration requires fewer iterations, each iteration includes policy evaluation, so its total computation time is longer than value iteration. For this problem size, value iteration is both simple to implement and efficient.

Q-learning requires many more episodes, but because it updates only visited states, the overall computation time remains comparable to value iteration.

B. Path Visualization

Figures 1, 2, and 3 show the paths obtained from VI, PI, and QL, respectively. Black cells are obstacles, white cells are free space, the blue line is the trajectory of the front endpoint of the mattress, and the red line is the trajectory of the back endpoint. The blue/red dots denote the start position and the green star denotes the goal position.

The three paths are nearly identical. The mattress first moves down a narrow vertical corridor, then enters the central corridor, moves into the left room, and finally rotates near the goal to align with the target orientation. The front and back

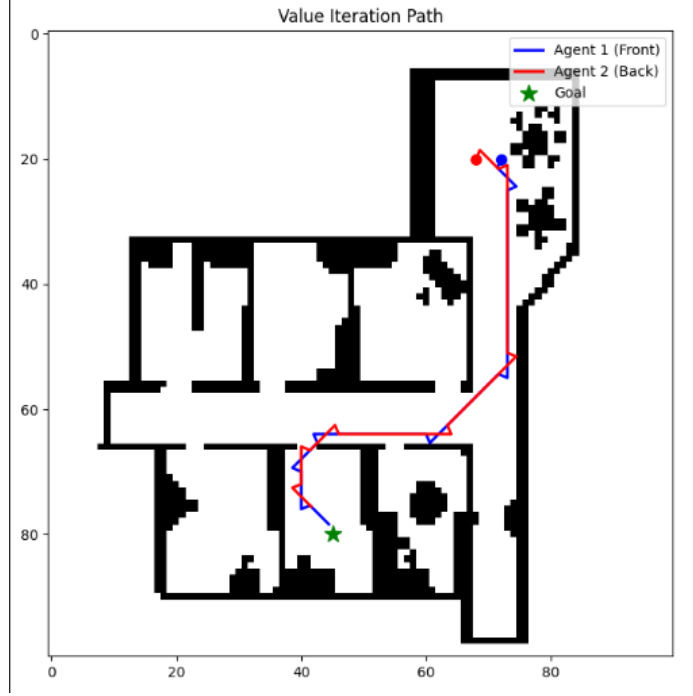


Fig. 1. Mattress path obtained by Value Iteration.

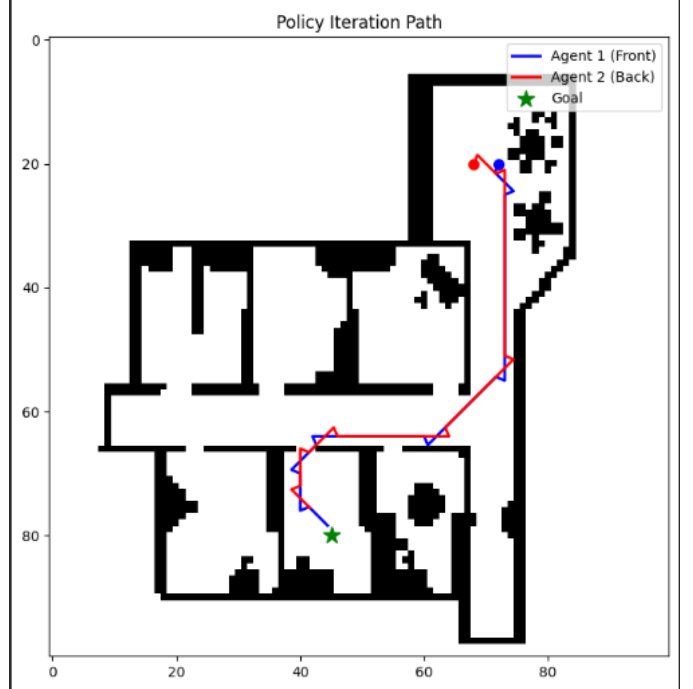


Fig. 2. Mattress path obtained by Policy Iteration.

trajectories both maintain clearance from obstacles, confirming that the collision checking logic that considers both endpoints works correctly.



Fig. 3. Mattress path obtained by Q-Learning.

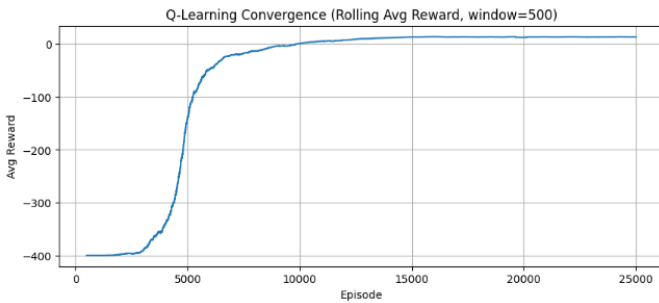


Fig. 4. Q-Learning convergence curve: 500-episode rolling average return.

C. Q-Learning Convergence Analysis

Figure 4 shows the 500-episode rolling average return of Q-learning. Initially, the average reward is around -400 , meaning that most episodes fail to reach the goal and terminate at the maximum step limit. Around 4,000–5,000 episodes, the curve rises sharply, indicating that the agent has discovered goal-directed behavior thanks to reward shaping. After about 10,000 episodes, the average reward approaches zero, and then slowly increases until 25,000 episodes as the policy is refined.

From the training log, the number of goal-reaching episodes is about 863 (approximately 17%) at 5,000 episodes, and increases to 20,777 (about 83%) by the end of 25,000 episodes.

VII. KEY CHALLENGES

In this section, we summarize the core challenging aspects highlighted in the task description and relate them to the implementation and experiments conducted in this project.

A. Complex State–Action Modeling

A key characteristic of this problem is that the state is not just a position (x, y) , but an augmented state (x, y, θ) that includes orientation. The mattress is modeled as a one-dimensional rigid body with a fixed length, and the state is defined by the center coordinates on the grid and one of eight discrete orientations (at 45-degree intervals).

Collision checking verifies not only the agent’s center cell but also the front and rear endpoints of the mattress, and a state is valid only if all three lie within the map and on free cells. This more realistic modeling of an extended body in narrow corridors and corners, however, requires additional endpoint computation and collision checks at every transition, making the state–action model more complex and increasing the computational cost compared to a point-robot problem.

B. Model-Based Computational Cost

Value Iteration (VI) and Policy Iteration (PI) assume that both the transition model and the reward function are known and compute the optimal policy via dynamic programming. In this project, instead of storing a full transition table, we traversed all valid states and actions and directly called the environment model (`env.step`) to obtain the next state and reward.

In the 100×100 grid with eight orientations and three actions, Value Iteration converged in a few dozen iterations but still required repeated sweeps over the entire state space, leading to noticeable computation time. Policy Iteration needed a similar number of outer iterations plus additional policy evaluation, resulting in even longer execution time. Thus, model-based methods generate paths very efficiently once a policy has been computed, but the initial computational effort grows rapidly with the size of the state space.

C. Model-Free Exploration and Sparse Rewards

Q-Learning is a model-free reinforcement learning algorithm that learns the Q-function purely through interaction with the environment, without explicit transition probabilities. The reward structure in this project is a typical sparse reward setting: a large positive reward is given only near the goal, and a small negative reward is given at every time step otherwise, so an almost random initial policy leads to many unsuccessful episodes and slow learning.

To mitigate this, we applied distance-based reward shaping using the change in distance to the goal: if the next state is closer, we add a small bonus; if it is farther, we add an additional penalty. This encourages the agent to move in goal-oriented directions, increases the rolling-average reward and the number of successful episodes, and eventually allows Q-Learning to learn a path of comparable quality to that obtained by VI/PI, emphasizing the importance of reward design and exploration strategy in model-free reinforcement learning.

D. The Core Trade-off: VI/PI vs. Q-Learning

This project illustrates the fundamental trade-off between model-based and model-free approaches. Model-based methods such as Value Iteration and Policy Iteration converge to

the optimal policy when the environment model is known and, in our experiments, produced (near-)optimal paths with very similar length and total reward; once the policy is available, generating a path from a given start state is computationally cheap, but computing the policy itself is expensive due to repeated sweeps over the entire state space.

In contrast, Q-Learning does not require explicit construction of the transition model and can be applied directly to environments that only provide a `step` interface, such as real robots or complex simulators. However, under a sparse reward structure, learning is very slow unless appropriate reward shaping and exploration scheduling are introduced, and many episodes may be required to reach a policy of comparable quality.

Overall, this project demonstrates that dynamic programming-based methods such as Value and Policy Iteration are powerful and efficient when a transition model is available, whereas model-free methods such as Q-Learning are more practical when the model is difficult to obtain and only interaction data are accessible, at the cost of longer training and careful reward and exploration design.

VIII. CONCLUSION AND FUTURE WORK

In this project, we formulated the problem of moving a long mattress in an indoor environment, derived from a floor plan image, as an MDP, and compared value iteration, policy iteration, and Q-learning as solution methods. Modeling the mattress as a rigid body of length five cells and performing collision checks on both endpoints allowed us to capture narrow corridors and corners in a realistic way.

Value iteration was the best compromise between simplicity, computation time, and convergence speed, while policy iteration was slower due to the cost of policy evaluation. Q-learning does not require an environment model, but in a sparse-reward setting it required distance-based reward shaping to learn effectively. With shaping, Q-learning converged to a path of similar quality to those from dynamic programming.

Future work includes: (1) extending the model to continuous states and orientations and introducing function approximation (e.g., deep Q-networks) to handle larger maps; (2) applying potential-based reward shaping theory to design shaping terms that preserve optimality in a more principled way; and (3) modeling the mattress as a multi-joint robotic arm to study how the size of the state space and the learning difficulty scale with degrees of freedom.

REFERENCES

- [1] J. T. Schwartz and M. Sharir, “On the ‘Piano Movers’ problem I. The case of a two-dimensional rigid polygonal body moving amidst polygonal barriers,” *Communications on Pure and Applied Mathematics*, vol. 36, pp. 345–398, 1983.
- [2] W. E. Red and H.-V. Truong-Cao, “The Configuration Space Approach to Robot Path Planning,” in *Proceedings of the American Control Conference*, 1984.
- [3] T. Bretl, S. Lall, J.-C. Latombe, and S. Rock, “Multi-Step Motion Planning for Free-Climbing Robots,” in *Algorithmic Foundations of Robotics VI*, Springer, 2005.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: The MIT Press, 2018.

- [5] L. Zhang, H. Xiong, O. Ma, and Z. Wang, “Multi-Robot Cooperative Object Transportation Using Decentralized Deep Reinforcement Learning,” arXiv preprint arXiv:2007.09243, 2020.

IX. INDIVIDUAL CONTRIBUTION

My primary focus was exploring the theoretical limits of the system through Value Iteration and Policy Iteration [4]. I wanted to understand the “ground truth” of optimal behavior before diving into learning-based methods, which led me to implement these model-based algorithms myself [4].

I was also particularly interested in how these algorithms handle realistic constraints, so I came up with the idea of generating our test environments directly from real-world floor plans [2]. This allowed me to test how the “mattress” navigates tight, non-trivial geometries [3].

In designing the MDP, I had to tackle the core questions of multi-agent coordination:

- (i) **Handling Actions:** I wondered how to synchronize two robots without complex communication protocols. I solved this by defining the actions for the coupled system rather than the individuals, in contrast to decentralized approaches [5]. Instead of translating global goals into specific wheel velocities for each robot, I modeled the action as a direct command to move the mattress itself (the rigid body) [1]. This abstraction implies the robots are perfectly coordinated to execute the move.
- (ii) **Information Access:** I also had to decide what the agents “see.” To make the Value and Policy Iteration work effectively, I assumed the system has access to the global state (x, y, θ) [4]. This means the planner isn’t guessing; it acts with full observability of the entire coupled configuration and the map [2].